

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I ____BAIRAGANI DINESH --192372189____ (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 192372189

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night)

such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

ANS:-

Backtracking Search – Doctor Shift Assignment

Aim:-

To assign **3 doctors (D1, D2, D3)** to **Morning, Afternoon, and Night** shifts using **Backtracking Search** while satisfying all constraints.

Constraints:-

- D1 cannot work **Night**.
- D2 must work **before** D3 (Morning < Afternoon < Night).
- Only **one doctor** per shift.
- D3 cannot work **Morning**.
- Every doctor is assigned **exactly one shift**.

Backtracking Steps:-

Step 1

Assign **D1 → Morning**

Remaining shifts: Afternoon, Night

Step 2

Try **D2 → Night**

Then D3 must be after Night, which is impossible.

Backtrack

Step 3

Assign **D2** → **Afternoon**

Remaining shift: Night

Step 4

Assign **D3** → **Night**

Check Constraints:

- D1 ≠ Night
- D2 before D3 (Afternoon < Night)
- D3 ≠ Morning
- One doctor per shift
- All doctors assigned

Final Valid Schedule:--

Doctor Shift

D1 Morning

D2 Afternoon

D3 Night

Result:-

A valid schedule is found using **Backtracking Search**.

Final Assignment:

- **Morning** → **D1**
- **Afternoon** → **D2**
- **Night** → **D3**

Conclusion::-

Backtracking Search systematically explores assignments and backtracks whenever a constraint is violated. The valid schedule obtained is:

D1 → Morning, D2 → Afternoon, D3 → Night.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G).

Obstacles block certain paths.

Constraints:

i. Movement allowed: Up, Down, Left, Right

ii. Cost of each move = 1

iii. Cannot pass through obstacles

iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

ANS:-

Since the grid is missing, here is a complete assignment answer using a standard 5×5 grid as an example.

BFS Search Algorithm – Robot Path Finding

Aim

To find the shortest path from Start (S) to Goal (G) using the Breadth-First Search (BFS) algorithm.

Assumptions (5×5 Grid)

```
S . . X .  
. X . X .  
. X . . .  
. . X X .
```

. . . . G

S = Start

G = Goal

X = Obstacle

. = Free Cell

Constraints

- Move only Up, Down, Left, Right
- Cost of each move = 1
- Cannot move through obstacles
- BFS explores nodes level by level

Step-by-Step Node Expansion

Step	Expanded Node	Queue Expansion	After
1	S	(1,0), (0,1)	
2	(1,0)	(0,1), (2,0)	
3	(0,1)	(2,0), (0,2)	
4	(2,0)	(0,2), (3,0)	
5	(0,2)	(3,0), (1,2)	
6	(3,0)	(1,2), (4,0), (3,1)	
7	(1,2)	(4,0), (3,1), (2,2)	
8	(4,0)	(3,1), (2,2), (4,1)	
9	(3,1)	(2,2), (4,1)	
10	(2,2)	(4,1), (2,3)	
11	(4,1)	(2,3), (4,2)	
12	(2,3)	(4,2), (2,4)	
13	(4,2)	(2,4), (4,3)	
14	(2,4)	(4,3), (3,4)	

15 (4,3) (3,4), G
16 G Goal Reached

Optimal Path

$S \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0)$
 $\rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow G$

Total Cost

Each move costs 1.

Total Cost = 8

Result

- Shortest Path: $S \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow G$
- Minimum Cost = 8

Conclusion

The Breadth-First Search (BFS) algorithm explores nodes level by level and guarantees the shortest path in an unweighted grid. Since every move has the same cost (1), BFS successfully finds the optimal path from Start (S) to Goal (G) while avoiding obstacles.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a

survivor located at G.

Constraints:

The robot can move up, down, left, right

Some cells are blocked (obstacles) and cannot be traversed

The robot has limited sensing capability (can only observe adjacent cells)

Each movement has a cost of 1, but entering risky zones has an additional cost of 2

The robot must minimize total cost while ensuring safe navigation

The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making

process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

Intelligent agents

Rationality

Environment type

d) Justify why your chosen approach is the most suitable for solving this problem

ANS:-

a) Analysis of Constraints & Impact on Decision-Making

- **Partially Observable Environment:** Because the robot can only observe adjacent cells, it cannot generate a complete global path from start to finish beforehand. Decisions must be made incrementally based on local perception.
- **Grid Movement (Up, Down, Left, Right):** Limits the action space at any given state to at most 4 orthogonal actions, simplifying local planning.
- **Blocked Cells (Obstacles):** Forces the robot to dynamically re-evaluate its path when an expected open cell turns out to be blocked upon arrival in an adjacent room.
- **Variable Path Costs (default, for risky zones):** Simple step-counting or Breadth-First Search (BFS) will fail to find the optimal path. The robot must account for path costs, balancing geometric distance against risk penalties.

- Online Search Requirement: The robot must interleave exploration (sensing/moving) with planning, building its internal map on the fly and backtracking when encountering dead ends or unexpectedly high-cost paths.

b) Solution Approach: Online Search Agent with Local Uniform Cost Planning

Because the robot does not know the full environment map in advance, standard offline algorithms like traditional or Uniform Cost Search (UCS) cannot be directly executed from start to finish. An Online Search Agent (such as LRTA*—Learning Real-Time, or an Online UCS / Dynamic Map Building Agent) is required.

Algorithmic Steps:

1. State Representation & Internal Map Initialization:
 - Maintain a local grid map where cells are marked as Unexplored, Free, Risky, or Blocked.
 - Keep track of the robot's current coordinates and the goal position.
2. Perception Phase (Sensing):
 - At the current cell, sense all adjacent cells (Up, Down, Left, Right).
 - Update the internal map with cell types and actual step costs:
 - Free cell step cost =
 - Risky cell step cost =
 - Obstacle = (impassable)
3. Local Path Planning & Action Selection:
 - Using the current known map, run a local Uniform Cost Search (or calculate estimated total cost using Manhattan distance as heuristic).
 - Select the adjacent unvisited or optimal cell that minimizes:
4. Execution & Movement:
 - Execute the selected action to move into the chosen adjacent room.
 - Increment the accumulated path cost ().
5. Backtracking & Loop Detection:
 - If all adjacent unvisited paths lead to obstacles or high-cost traps, use previously stored path history to backtrack to the nearest unvisited node with the lowest estimated cost.

6. Termination:

- Repeat steps 2–5 until the robot reaches survivor location .

c) Application of Core AI Concepts

1. Intelligent Agent Formulation

- PEAS Description:
 - Performance Measure: Minimize total path cost (movement + risk penalties), reach survivor , ensure zero collisions.
 - Environment: Disaster-affected grid building with unknown obstacles and risky zones.
 - Effectors: Actuators/wheels to move Up, Down, Left, Right.
 - Sensors: Proximity/visual sensors to detect terrain type and obstacles in adjacent cells.

2. Rationality

- A rational agent chooses actions that maximize its expected performance measure given its percept sequence and built-in knowledge.
- Here, rationality means the robot never moves into a known higher-cost cell (e.g., a risky zone with cost) if a safe cell (cost) offers a viable path toward , nor does it repeat unproductive loops.

3. Environment Classification

- Partially Observable: The robot sees only adjacent cells, not the full grid.
- Dynamic / Semi-static: The physical building topology is static, but from the agent's perspective, knowledge updates dynamically (online).
- Stochastic / Deterministic: Deterministic (movement actions succeed reliably without slip).
- Sequential: Current choices directly affect future available positions and total path cost.
- Discrete: Grid-based states, distinct time steps, and discrete actions.
- Single-Agent: Only the rescue robot is planning and taking action.

d) Justification of the Approach

Requirement	Why Online Agent / Local UCS Fits Best
-------------	--

Partial Observability	Offline algorithms require complete map data before execution. An Online Search Agent is explicitly designed to handle unexplored environments by interleaving planning and acting.
Non-Uniform Costs	Greedy/BFS approaches ignore path costs. Integrating Uniform Cost Search principles ensures the robot evaluates step costs (vs), avoiding high-risk zones unless necessary.
Optimality & Completeness	Adopting an admissible heuristic (like Manhattan distance on a grid) within an online framework ensures the robot completes its search without getting permanently stuck in loops.